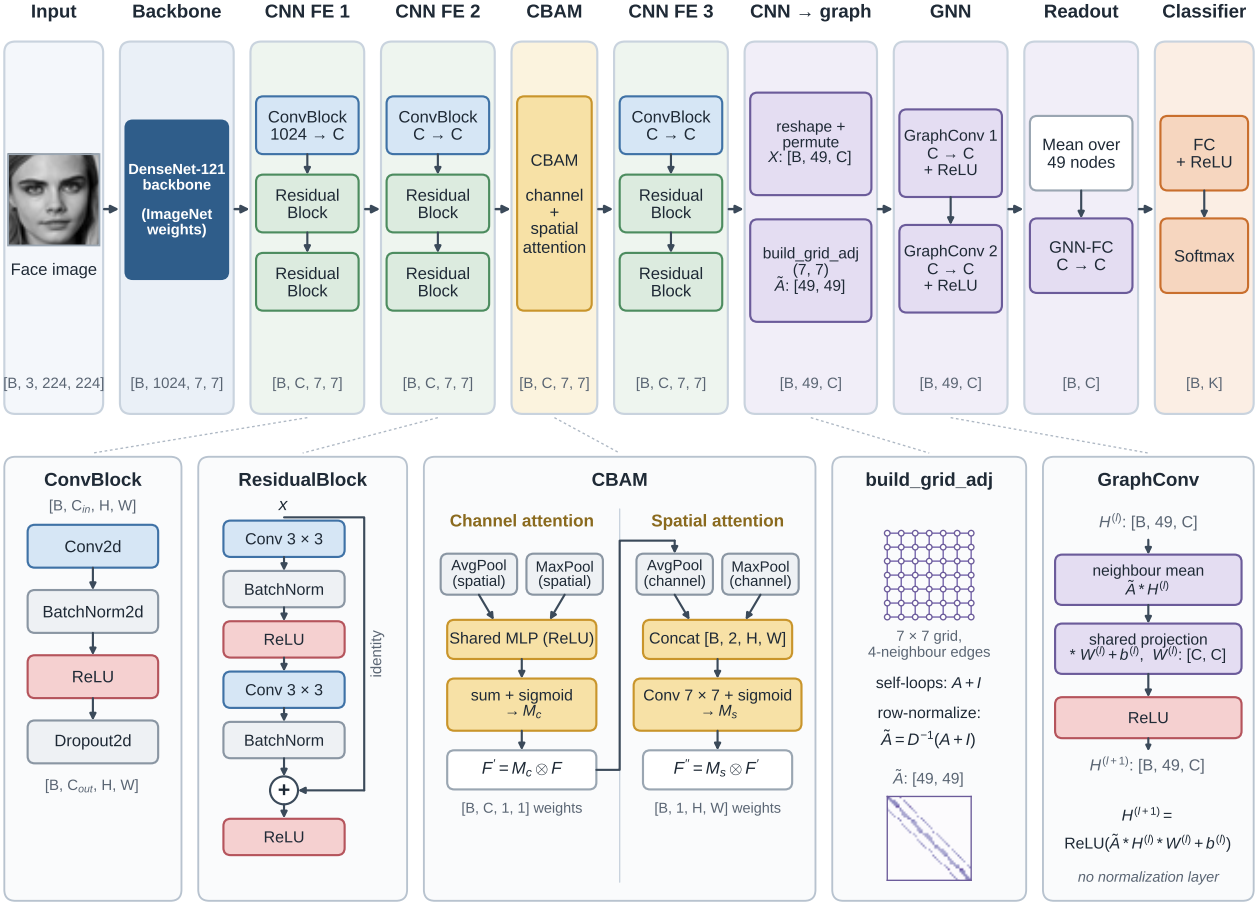

Input
Backbone
CNN FE 1
CNN FE 2
CBAM
CNN FE 3
CNN → graph
GNN
Readout
Classifier
Face image
[B, 3, 224, 224]
DenseNet-121 backbone (ImageNet weights)
[B, 1024, 7, 7]
ConvBlock 1024 → C
Residual Block
Residual Block
[B, C, 7, 7]
ConvBlock C → C
Residual Block
Residual Block
[B, C, 7, 7]
CBAM
channel + spatial attention
[B, C, 7, 7]
ConvBlock C → C
Residual Block
Residual Block
[B, C, 7, 7]
reshape + permute
X: [B, 49, C]
build_grid_adj (7, 7)
Ã: [49, 49]
[B, 49, C]
GraphConv 1 C → C + ReLU
GraphConv 2 C → C + ReLU
[B, 49, C]
Mean over 49 nodes
GNN-FC C → C
[B, C]
FC + ReLU
Softmax
[B, K]
ConvBlock
[B, C_in, H, W]
Conv2d
BatchNorm2d
ReLU
Dropout2d
[B, C_out, H, W]
ResidualBlock
x
Conv 3 × 3
BatchNorm
ReLU
Conv 3 × 3
BatchNorm
identity
ReLU
CBAM
Channel attention
AvgPool (spatial)
MaxPool (spatial)
Shared MLP (ReLU)
sum + sigmoid → M_c
F' = M_c ⊗ F
[B, C, 1, 1] weights
Spatial attention
AvgPool (channel)
MaxPool (channel)
Concat [B, 2, H, W]
Conv 7 × 7 + sigmoid → M_s
F'' = M_s ⊗ F'
[B, 1, H, W] weights
build_grid_adj
7 × 7 grid, 4-neighbour edges
self-loops: A + I
row-normalize:
Ã = D^{-1}(A + I)
Ã: [49, 49]
GraphConv
H^(l): [B, 49, C]
neighbour mean Ã * H^(l)
shared projection * W^(l) + b^(l), W^(l): [C, C]
ReLU
H^(l+1): [B, 49, C]
H^(l+1) = ReLU(Ã * H^(l) * W^(l) + b^(l))
no normalization layer